

Unit 4 Greedy Algorithms

Greedy Paradigm is straight forward way of algorithm design and generally solves optimization problems. In greedy programming: Input elements are exposed to some constraints to get the feasible solutions and the feasible solution that meets the objective function best among all solutions is called optimal solution. **Greedy algorithms always make local optimal choice in the hope that, it will generate global optimal solution.** However, it is not guaranteed that all greedy algorithms yield optimal solutions.

Most of the problems that can be solved using greedy approach have two parts:

a) **Greedy choice property** we can assemble a globally optimal solution by making locally optimal (greedy) choices.

b) **Optimal substructure** A problem exhibits optimal substructure if an optimal solution to the problem contains within it, optimal solutions to the sub-problems.

To prove that a greedy algorithm is optimal we must show the above two parts are exhibited by the problem.

1 . Fractional-Knapsack problem Statement:

A thief has a bag or knapsack that can contain maximum weight **W** of his loot. There are **n** items and the weight of i^{th} item is w_i and it worth v_i . Any amount of item can be put into the bag i.e. x_i fraction of item can be collected, where $0 \leq x_i \leq 1$. Here the objective is to collect the items that maximize the total profit earned.

Algorithm:

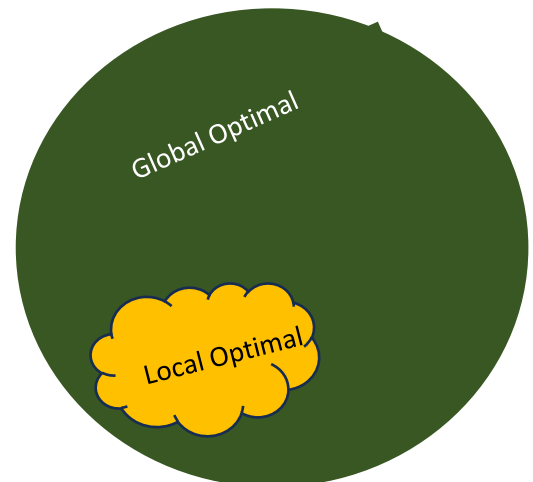
Take as much of the item with the highest value per weight (v_i/w_i) as you can. If the item is finished then move on to next item that has highest value per weight, continue this until the knapsack is full.

Input: $v[1, 2, \dots, n]$ and $w[1, 2, \dots, n]$ contain the values and weights respectively of the n objects sorted in decreasing order of (v_i/w_i). W is the capacity of the knapsack.

Output: $x[1, 2, \dots, n]$ is the solution vector that contains fractional amount of n items.

GreedyFracKnapsack(W, n)

```
{
    for(i=1; i<=n; i++)
        x[i] = 0.0;
    tempW = W;
    for(i=1; i<=n; i++)
    {
        if(w[i] <= tempW)
        {
            x[i] = 1.0;
            tempW = tempW - w[i];
        }
        else x[i] = tempW/w[i];
    }
    return x;
}
```



Analysis: To sort items into decreasing order of $(v_i/w_i) = O(n \log n)$ and for loop takes $O(n)$

$T(n) = O(n \log n) + O(n) = O(n \log n)$

Example: Given weight-value pair of items, first sort items on decreasing order of (v_i/w_i) and then just put first item into the bag, when it is finished move to next and next and next.

Let the problem instance be with 3 items where $v[] = \{10, 20, 30\}$ and $w[] = \{60, 100, 120\}$ and $W = 150$

2. Job sequencing with Deadlines,

Arrangement of jobs on a single processor with deadline constraints is named as job sequencing with deadlines. We arrange n jobs in sequence to obtain maximum benefit.

Statement:

- Each i th job is associated with a deadline $d_i \geq 0$ and profit $p_i > 0$.
- For any i th job, the profit p_i is earned, if job is completed by its deadline.
- Each job takes one-unit of time to complete.
- Only one processor is available for processing all jobs

Feasible solution to this problem is a subset list of jobs such that each job in this subset can be completed by its deadline. A value of feasible solution is the sum of the profits of the jobs in the list ($\sum p_i$).

Example: Consider a sequencing problem where 4 jobs have a profit of (10, 30, 60, 40) and corresponding deadlines (2, 3, 1, 3).

Brute-Force approach: List of feasible solutions are:

S.n	Feasible Sol ⁿ list	Processing sequence	Total profit $\sum_{i \in J} p_i$
1	{1,2}	1→2	40
2	{1,3}	3→1	70
3	{1,4}	1→4	50
4	{2,3}	3→2	90
5	{2,4}	2→4 or 4→2	70
6	{3,4}	3→4	100
7	{1,2,3}	3→1→2	100
8	{1,2,4}	1→2→4 or 2→1→4 or 4→1→2	80
9	{1,3,4}	3→1→4	110
10	{2,3,4}	3→2→4 or 3→4→2	130
11	{1}	1	10
12	{2}	2	30
13	{3}	3	60
14	{4}	4	40

Solution 10 is optimal yielding a total profit of 130. Optimal job sequence is either (3, 2, 4) or (3, 4, 2).

2.1 . Greedy approach: It formulates an optimization measure (objective function ($\sum p_i$)) to determine how next job is selected. What it says is: next job to include is that increases total profit the most, which requires considering jobs in decreasing order of p_i 's.

```

JOBSEQUENCE(job[1..n], p[1..n], d[1..n])
{ Create list[1..n] and initialize all items to 0
profit = 0
for i = 1 to n {
    k = d[i]
    while(k > 0)
        { if(list[k] = 0)
            list[k] = job[i]
            profit += p[i]
          k--}
    } return list
}

```

Time complexity: Sorting (p_i into decreasing order) takes $O(n \log n)$ at worst. Selection of jobs takes constant time ($O(1)$). Consider n jobs in turn. For each job, inserting the job into the partial solution using its deadline takes $O(n)$ time. Hence, total running time: $O(n^2)$.

For same example,

4 jobs have a profit of (10, 30, 60, 40) and corresponding deadlines (2, 3, 1, 3).

ordering jobs in decreasing order of profit;

Profits = (60, 40, 30, 10)

Job # = (3, 4, 2, 1)

Deadlines = (1, 3, 3, 2)

Job (i)	p_i	d_i	Profit	Operation	List[1..4] initialized to 0
3	60	1	60	Assign job 3 to slot 1	3 0 0 0
4	40	3	100	Assign job 4 to slot 3	3 0 4 0
2	30	3	130	Assign job 2 to slot 2 because 3 is not empty	3 2 4 0
1	10	2	130	Reject job 1, because it violates its deadline.	3 2 4 0

Optimal job sequence is 3→4→2 with maximum profit of 130;

3. Huffman codes

It compresses data very efficiently: saves up to 20-90%. Assuming data to be sequence of characters, Huffman greedy algorithm uses character frequency table to build up an optimal way of representing each character as a binary string.

Let's start with example: Considering 100000-character file with 6 repeating characters with frequencies:

Characters	a	b	c	d	e	f
Frequency in thousands	45	13	12	16	9	5

Many options to represent such file of information with binary character code:

Fixed-Length codeword: we need 3-bits to represent 6 characters; a=000, b=001,..., f=101, requiring 300000 bits to code entire file. Can it be better? Of course, with following option.

Variable-Length codeword: Strategy is to use short codeword for frequent-characters and longer for infrequent. Suppose, we encode like one given in following table. We'll notice the difference.

Characters	a	b	c	d	e	f
Frequency in thousands	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

Fixed –length codeword Requires 300000 bits to store file.

Variable –length codeword Requires 224000 bits, saves 25%.

Constructing Huffman codes:

It uses the notion of **prefix codes**; codes in which no codeword is also a prefix of some other codeword. For example: 001011101 is parsed uniquely as “aabe” using above variable-length scheme.

```

HUFFMAN(C)           // Set of characters
1  n = |C|           // Num of characters
2  Q = C             / Heap/priority queue/min heap
3  for i = 1 to n - 1
4      allocate a new node z
5      z.left = x = EXTRACT-MIN(Q)
6      z.right = y = EXTRACT-MIN(Q)
7      z.freq = x.freq + y.freq
8      INSERT(Q, z)           // log n
9  return EXTRACT-MIN(Q)    // return the root of the tree

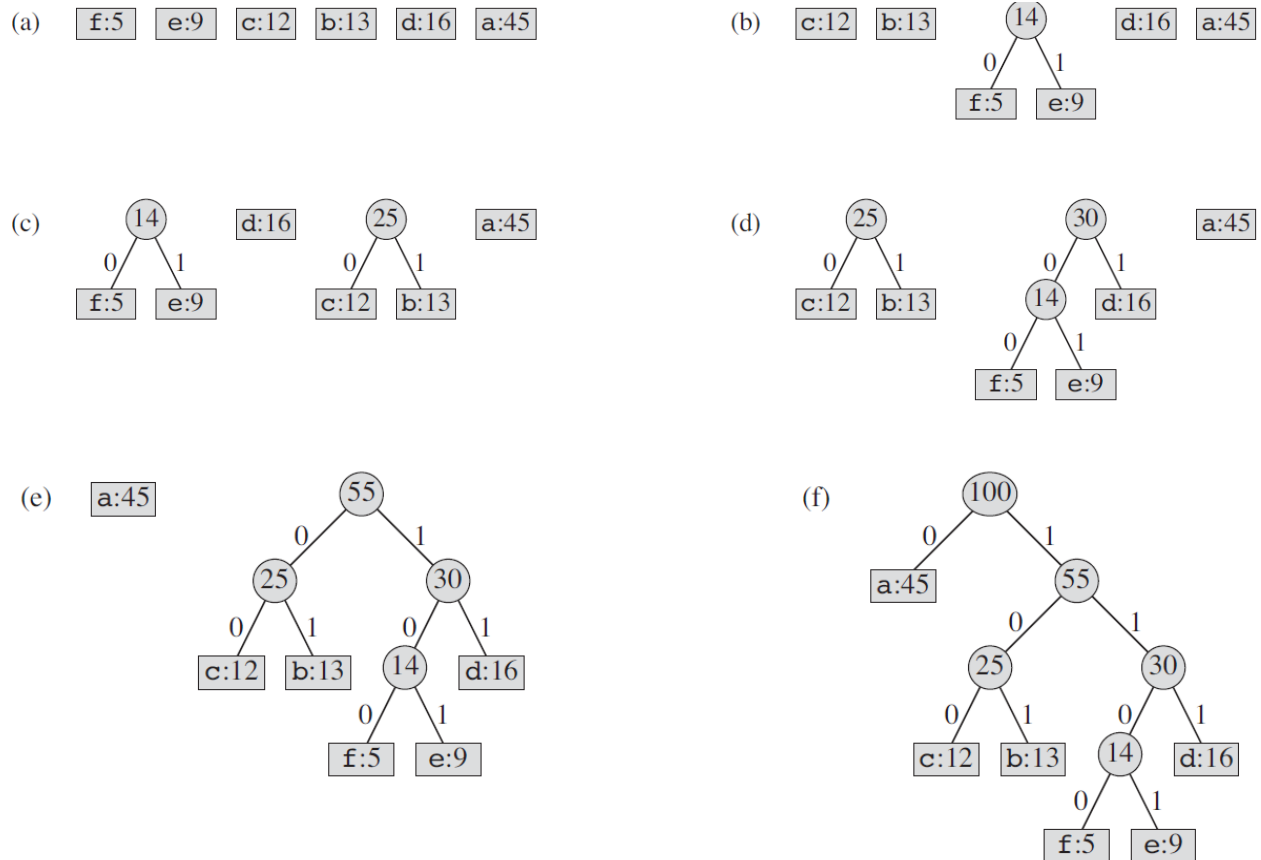
```

A min-priority queue Q, keyed on frequency f, is used to identify the two least-frequent objects to merge together. The result of the merger of two objects is a new object whose frequency is the sum of the frequencies of the two objects that were merged.

Analysis: For a set C of n characters, the initialization $Q \leftarrow C$: O(n) time using the BUILD-MINHEAP to maintain priority queue. The for loop is executed exactly n-1 times, and since each

heap operation requires time $O(\log n)$, the loop contributes $O(n \log n)$ to the running time. Thus, the total running time of HUFFMAN on a set of n characters is $O(n \log n)$.

Construction of Huffman tree for above example



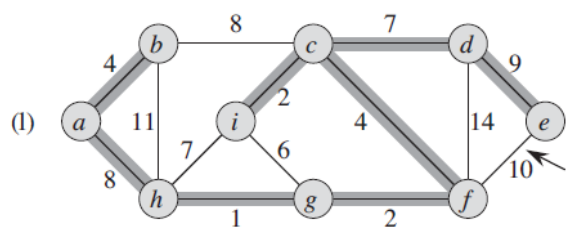
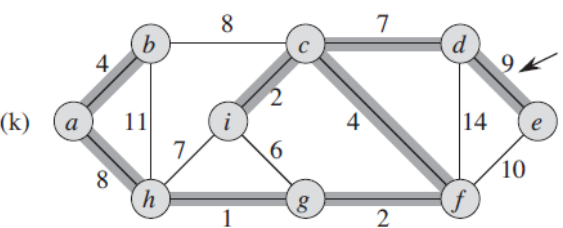
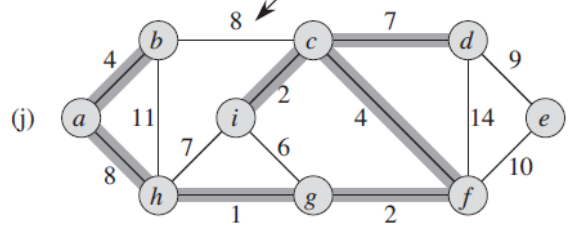
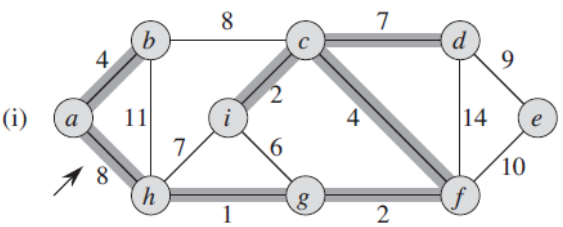
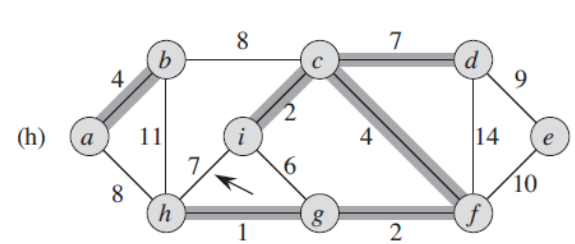
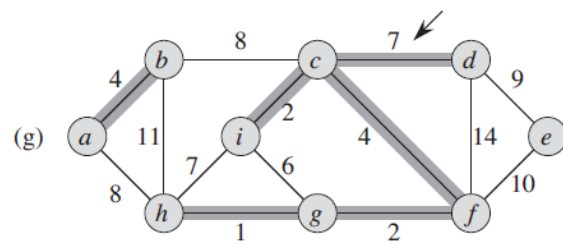
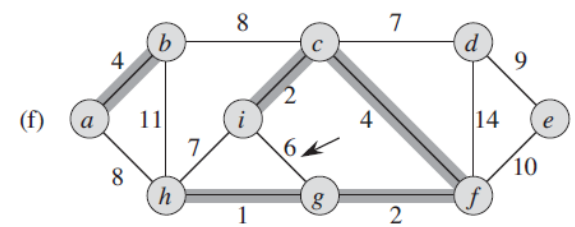
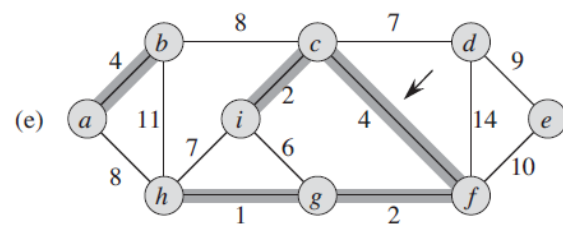
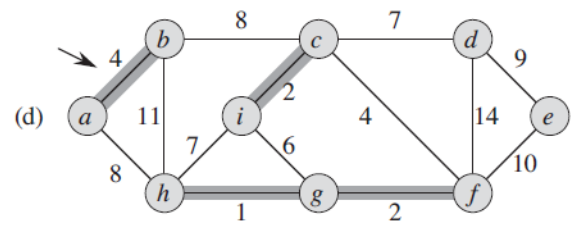
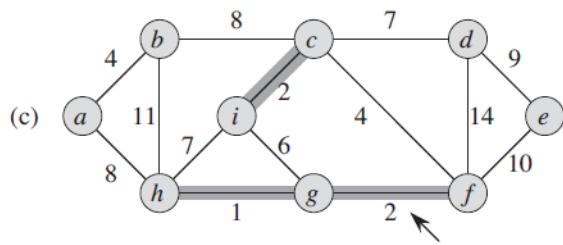
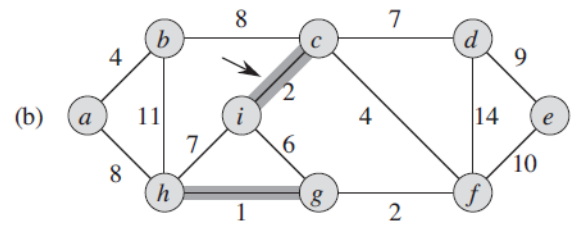
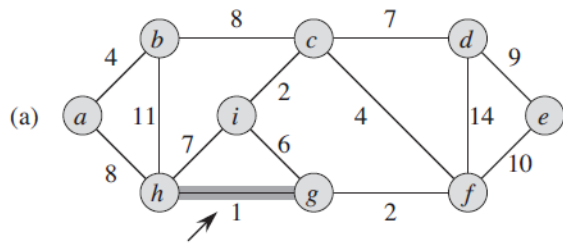
Note:

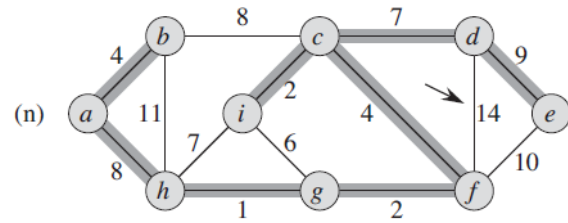
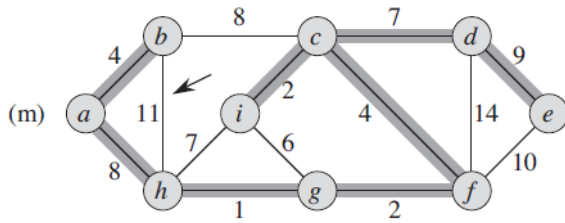
*An optimal code for a file is always represented by a full binary tree, in which every non-leaf node has two children, why ?
if C is the alphabet from which the characters are drawn and all character frequencies are positive, then the tree for an optimal prefix code has exactly $|C|$ leaves, one for each letter of the alphabet, and exactly $|C|-1$ internal nodes*

Kruskal's algorithm

The problem of finding MST can be solved by using Kruskal's algorithm. The idea behind this algorithm is to put the set of edges from the given graph $G = (V, E)$ in nondecreasing order of their weights. The selection of each edge in sequence then guarantees that the total cost of subgraph (MST) would be minimum. Note that we have G as a graph, V as a set of n vertices and E as set of edges of graph G .

Example: Find MST and its weight of the given graph





Algorithm:

KruskalMST(G)

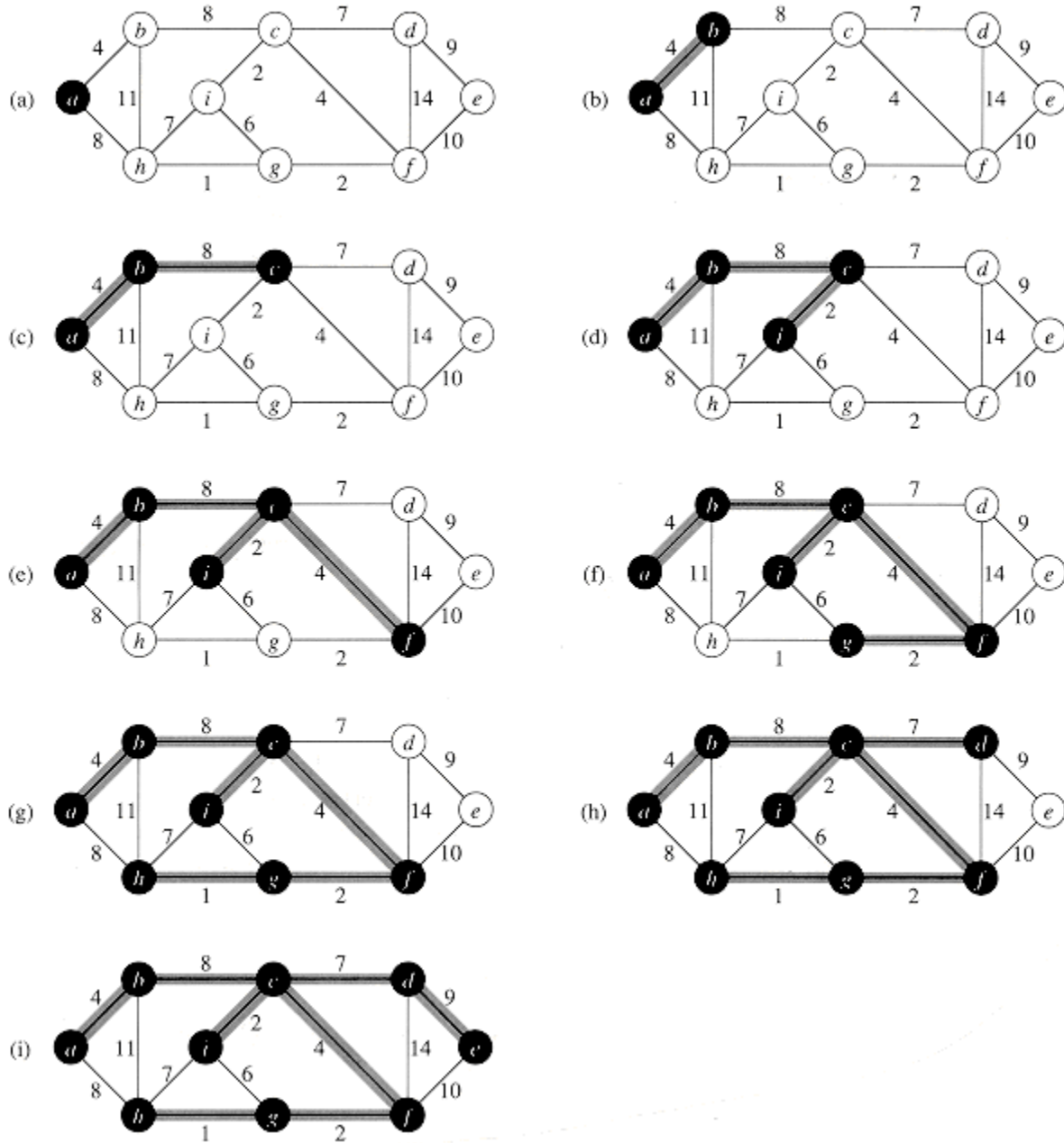
```
{
    T = {V} // forest of |V| nodes
    S = {E} // set of edges sorted in nondecreasing order of weight
    while( |T| < |V| - 1 and S != Φ )
    {
        Select (u, v) from S in order
        Remove (u, v) from E
        if((u, v) does not create a cycle in T) // find whether u and v belongs to different set
            T = T ∪ {(u, v)} // union
    }
}
```

Analysis:

In the above algorithm the n tree forest at the beginning takes $|V|$ time, the creation of set S takes $O(|E|\log|E|)$ time and while loop execute $O(|V|)$ times and the steps inside the loop take almost linear time (see disjoint set operations; find and union). So the total time taken is $O(|E|\log|E|)$ or asymptotically equivalent to $O(|E|\log|V|)$.

See Disjoint set and Operations(find(),union()) operations for implementation purpose

Prims Algorithm



The execution of Prim's algorithm on the graph from Figure. The root vertex is a. Shaded edges are in the tree being grown, and the vertices in the tree are shown in black. At each step of the algorithm, the vertices in the tree determine a cut of the graph, and a light edge crossing the cut is added to the tree. In the second step, for example, the algorithm has a choice of adding either edge (b, c) or edge (a, h) to the tree since both are light edges crossing the cut.

PrimMST(G)

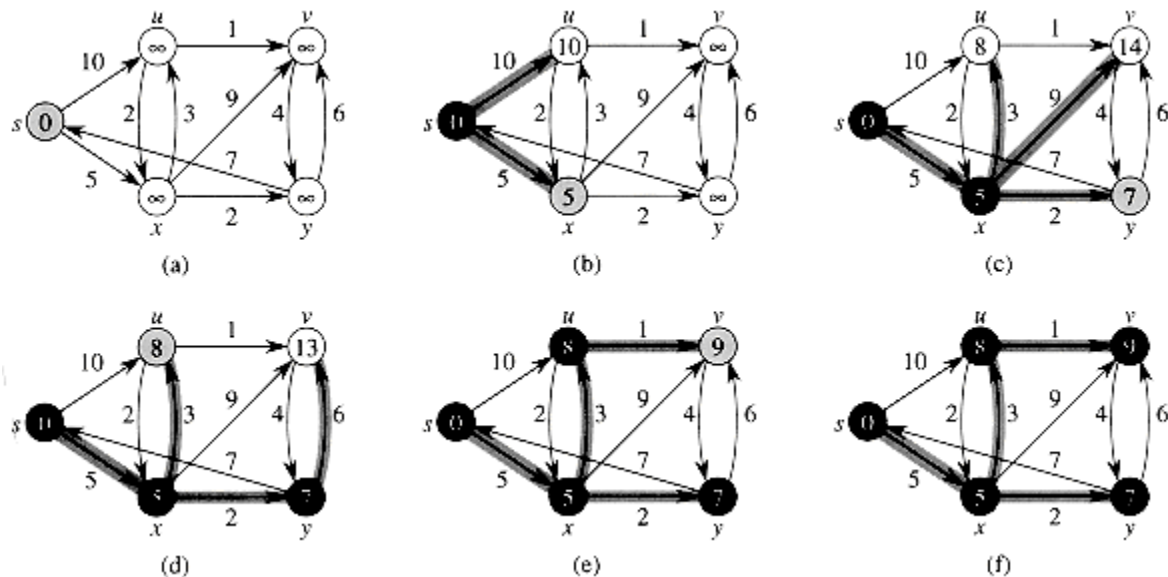
```
{
    T =  $\phi$ ; // T is a set of edges of MST
    S = {s} ; //s is randomly chosen vertex and S is set of vertices
    while(S != V)
    {
        e = (u,v) an edge of minimum weight incident to vertices in T and not
        forming a
        simple circuit in T if added to T i.e.  $u \in S$  and  $v \in V-S$ 
        T = T  $\cup$  {(u,v)};
        S = S  $\cup$  {v};
    }
}
```

Analysis:

In the above algorithm while loop execute $O(V)$. The edge of minimum weight incident on a vertex can be found in $O(E)$, so the total time is $O(EV)$. We can improve the performance of the above algorithm by choosing better data structures as priority queue and normally it will be seen that the running time of prim's algorithm is $O(E \log V)$.

Dijkstra's Algorithm and their Analysis

Dijkstra's algorithm solves the single-source shortest-paths problem on a weighted, directed graph $G = (V, E)$ for the case in which all edge weights are nonnegative.



```

Dijkstra(G,w,s) // weighted graph, with starting vertex s
{
  for each vertex v ∈ V
    do d[v] = ∞ //unknown distance from s to v
    d[s] = 0    // distance from s to s
    S = ∅ // set of all visited vertices
    Q = V //set of all unvisited vertices
  While(Q ≠ ∅)
  {
    u = Take minimum from Q and delete.
    S = S ∪ {u} // make it visited
    for each vertex v adjacent to u
      do if d[v] > d[u] + w(u,v) // relax (u,v)
         then d[v] = d[u] + w(u,v)
  }
}

```

Analysis:

In the above algorithm, the first for loop block takes $O(V)$ time. Initialization of priority queue Q takes $O(V)$ time. The while loop executes for $O(V)$, where for each execution the block inside the loop takes $O(V)$ times. Hence the total running time is $O(V^2)$.

Assignment

Correctness of Algorithms(Study)

Way of giving argument that the algorithm works correctly for any instance of input.

By Induction for recursive algorithm.

Three Conditions (initialization, maintenance, termination) for iterative algorithms.